



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251213
Nama Lengkap	Maryo Aurel Nubatonis
Minggu ke / Materi	11 / Tipe Data Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1

Dictionary

Pengertian dan perbandingan Dictionary memiliki kemiripan dengan list, namun lebih bersifat umum. Perbedaan utama adalah pada indeksnya jika list harus menggunakan indeks berupa integer, dictionary memungkinkan indeks (yang disebut kunci) berupa tipe data apa pun.

Struktur data adalah anggota dictionary terdiri dari pasangan kunci:nilai atau sering disebut sebagai item. Dictionary berfungsi sebagai pemetaan dimana setiap kunci unik memetakan ke suatu nilai tertentu.

Ketentuan kunci dalam sebuah dictionary harus bersifat unik, artinya tidak boleh ada dua kunci yang sama dalam satu dictionary yang sama.

Contoh penerapan adalah salah satu contoh penggunaannya adalah pembuatan kamus bahasa, seperti memetakan kata-kata bahasa inggris (sebagai kunci) ke bahasa Spanyol (sebagai nilai).

Cara pembuatan untuk membuat dictionary kosong, gunakan fungsi bawaan dict(). Karena dict adalah fungsi internal Python, penggunaannya harus dihindari sebagai nama variable agar tidak terjadi konflik.

```
>>> ens2sp = dict()
>>> print(ens2sp)
{}

```

Tanda kurung kurawal { } digunakan untuk membuat dictionary kosong, sedangkan kurung kotak [] digunakan untuk menambahkan item pada dictionary.

```
>>> ens2sp['one'] = 'uno'
```

Pada potongan kode tersebut, item dictionary dibuat dengan kunci 'satu' dan nilai 'uno'. Saat dictionary dicetak, akan terlihat pasangan antara kunci dan nilainya.

```
>>> print(ens2sp)
{'one': 'uno'}
```

Potongan kode tersebut membuat item dictionary dengan kunci 'satu' dan nilai 'uno'. Ketika dictionary ditampilkan, akan terlihat pasangan antara kunci dan nilainya.

```
>>> ens2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(ens2sp)
{'one': 'uno', 'two': 'dos', 'three': 'tres'}
```

Urutan pasangan kunci dan nilai pada dictionary tidak selalu sama atau tidak dapat diprediksi. Namun, hal ini bukan masalah karena dictionary menggunakan kunci untuk mencari nilai, bukan indeks angka.

```
>>> print(ens2sp['two'])
dos
```

Kunci 'two' selalu berpasangan dengan nilai "dos", sehingga urutan item pada dictionary tidak terlalu berpengaruh. Jika menggunakan kunci yang tidak ada di dictionary, maka akan muncul pengeculian (error).

```
>>> print(ens2sp['four'])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'four'
```

Fungsi `len` pada *dictionary* digunakan untuk menghitung jumlah pasangan kunci dan nilai.

```
>>> len(ens2sp)
3
```

Operator `in` pada *dictionary* digunakan untuk mengecek apakah suatu kunci ada di dalam *dictionary* dan akan menghasilkan nilai `True` atau `False`.

```
>>> 'one' in ens2sp
True
>>> 'uno' in ens2sp
False
```

Method `values()` pada *dictionary* digunakan untuk mengambil semua nilai pada *dictionary* dalam bentuk *list* dan dapat digunakan bersama operator `in`.

```
>>> vals = list(ens2sp.values())
>>> 'uno' in vals
True
```

Operator `in` pada *list* dan *dictionary* menggunakan algoritma yang berbeda. Pada *list* digunakan pencarian linear sehingga semakin banyak data, waktu pencarian semakin lama. Sedangkan pada *dictionary* digunakan algoritma Hash Table sehingga proses pencarian tetap cepat meskipun jumlah item bertambah.

MATERI 2

Dictionary sebagai set penghitung (counters)

Sebagai contoh, diberikan sebuah string dan diminta untuk menghitung jumlah kemunculan setiap huruf. Ada beberapa cara yang dapat digunakan untuk menyelesaikannya.

- Membuat 26 variabel untuk setiap huruf alfabet, lalu menggunakan kondisi berantai untuk mengitung kemunculan tiap huruf.
- Membuat list berisi 26 elemen, mengubah karakter menjadi angka sebagai indeks, kemudian menambah hitungan pada indeks tersebut.
- Membuat dictionary dengan huruf sebagai kunci dan jumlah kemunculan sebagai nilai, lalu memperbarui nilai setiap kali huruf muncul.

Dari tiga opsi tersebut, perhitungan yang dilakukan sebenarnya sama, tetapi setiap opsi menggunakan metode yang berbeda dalam prosesnya.

Implementasi dilakukan menggunakan perhitungan (computation), di mana beberapa metode dapat lebih efisien dibanding lainnya. Penggunaan dictionary lebih praktis karena tidak perlu mengetahui huruf apa saja yang akan muncul dalam string. Cukup menyediakan ruang untuk menampung huruf yang muncul. Model penerapannya sebagai berikut:

```

1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     if c not in d:
5         d[c] = 1
6     else:
7         d[c] += 1
8 print(d)

```

Model komputasi tersebut disebut histogram, yaitu istilah dalam statistika untuk menunjukkan frekuensi kemunculan suatu data.

Perulangan for digunakan untuk menelusuri setiap karakter dalam string. Jika karakter c belum ada di dictionary, maka akan dibuat item baru dengan kunci c dan nilai 1. Jika sudah ada, maka nilai pada kunci tersebut akan ditambah.

Output program diatas:

```
{'b': 1, 'r': 3, 'o': 3, 'n': 1, 't': 1, 's': 3, 'a': 1, 'u': 3}
```

Histogram output menunjukkan bahwa huruf "a" dan "b" muncul sekali; "o" muncul dua kali, dan seterusnya.

Dictionary memiliki method `get()` yang digunakan untuk mengambil nilai Berdasarkan kunci dengan nilai default. Jika kunci ada di dalam dictionary, maka akan mengembalikan nilai tersebut; jika tidak, akan mengembalikan nilai default yang ditentukan.

```
>>> counts = {'chuck': 1, 'annie': 42, 'jan': 100}
>>> print(counts.get('jan', 0))
100

>>> print(counts.get('tim', 0))
0
```

Method `get()` dapat digunakan untuk membuat loop histogram menjadi lebih ringkas. Method ini secara otomatis menangani kasus ketika kunci belum ada dalam dictionary, sehingga tidak perlu menggunakan `if`. Dengan demikian, beberapa baris kode dapat disederhanakan menjadi satu baris saja.

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     d[c] = d.get(c, 0) + 1
5 print(d)
```

Output:

```
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

Penggunaan method `get()` untuk menyederhanakan loop penghitung sudah menjadi idiom yang umum dalam python. Dibandingkan dengan penggunaan `if` dan operator `in`, kedua cara tersebut menghasilkan hasil yang sama, namun `get()` lebih ringkas dan efisien dalam penulisan kode.

MATERI 2

Dictionary dan File

Salah satu kegunaan dictionary adalah untuk menghitung jumlah kemunculan data dalam sebuah teks. Contohnya dapat dilakukan pada file sederhana yang diambil dari teks Romeo and Juliet.

Sebagai contoh pertama, akan digunakan versi teks yang disingkat dan disederhanakan tanpa tanda baca. Selanjutnya akan digunakan teks adegan dengan tanda baca yang disertakan.

But soft what light through yonder window breaks

It is the east and Juliet is the sun

Arise fair sun and kill the envious moon

Who is already sick and pale with grief

Program Python dapat dibuat untuk dibuat untuk membaca setiap baris pada file, memisahkan baris menjadi daftar kata, lalu menghitung jumlah kemunculan tiap kata menggunakan dictionary. Proses ini menggunakan nested loop, yaitu perulangan luar untuk membaca baris file dan perulangan dalam untuk memeriksa setiap kata pada baris tersebut. Dengan kombinasi kedua perulangan tersebut, semua kata dalam file dapat dihitung kemunculannya.

```

1  fname = input("Enter file name: ")
2
3  try:
4      fhand = open(fname)
5  except:
6      print("File cannot be opened:", fname)
7      exit()
8
9  counts = {}
10
11 for line in fhand:
12     words = line.split()
13
14     for word in words:
15         if word not in counts:
16             counts[word] = 1
17         else:
18             counts[word] += 1
19
20 print(counts)

```

Pada statement else, digunakan penulisan singkat untuk menambahkan nilai variable. Counts[word] += 1 memiliki arti yang sama dengan counts[word] = counts[word] + 1. Selain itu, terdapat operator lain seperti -=, *=, dan /= untuk mengubah nilai variable. Saat program dijalankan, hasil yang ditampilkan berupa jumlah kata dalam urutan hash yang belum terurut.

```

Enter file name: romeo.txt
{'But': 1, 'soft': 1, 'what': 1, 'light': 1, 'through': 1, 'yonder': 1, 'window': 1, 'breaks': 1, 'It': 1, 'is': 3, 'the': 3, 'east': 1, 'and': 3, 'Juliet': 1, 'sun': 2, 'Arise': 1, 'fair': 1, 'kill': 1, 'envious': 1, 'moon': 1, 'Who': 1, 'already': 1, 'sick': 1, 'pale': 1, 'with': 1, 'grief': 1}

```

MATERI 3

Looping dan Dictionary

Pada statement for, dictionary akan menelusuri setiap kunci dan menampilkan pasangan kunci beserta nilainya.

```
1 counts = {'chuck': 1, 'annie': 42, 'jan': 100}
2 for key in counts:
3     print(key, counts[key])
```

Outputnya :

```
chuck 1
annie 42
jan 100
```

Output menunjukkan bahwa kunci pada dictionary tidak memiliki urutan tertentu. Dengan looping, kita dapat mencari entri tertentu, misalnya nilai yang lebih dari 10.

```
1 counts = {'chuck': 1, 'annie': 42, 'jan': 100}
2 for key in counts:
3     if counts[key] > 10:
4         print(key, counts[key])
```

Outputnya :

```
annie 42
jan 100
```

Program diatas hanya akan menampilkan data nilai diatas 10.

Untuk menampilkan kunci secara alfabet, kunci pada dictionary di ubah menjadi list menggunakan method keys(), kemudian diurutkan dengan sort(), lalu dilakukan looping untuk menampilkan pasangan kunci dan nilainya secara terurut.

```
1 counts = {'chuck': 1, 'annie': 42, 'jan': 100}
2
3 lst = list(counts.keys())
4 print(lst)
5
6 lst.sort()
7
8 for key in lst:
9     print(key, counts[key])
```

Outputnya :

```
['chuck', 'annie', 'jan']
annie 42
chuck 1
jan 100
```

Pertama, ditampilkan list kunci yang belum terurut dari method keys(). Setelah itu, pasangan kunci dan nilai ditampilkan secara berurutan menggunakan loop for.

MATERI 3

Advanced Text Parsing

Pada bagian sebelumnya digunakan file tanpa tanda baca. Pada bagian ini digunakan file Romeo.txt dengan tanda baca lengkap untuk contoh pengolahan data.

But soft! what light through yonder window breaks?

It is the east and Juliet is the sun

Arise fair sun and kill the envious moon

Who is already sick and pale with grief

Python memiliki fungsi `split()` yang memisahkan string Berdasarkan spasi menjadi beberapa kata (token). Misalnya “soft” dan “soft” di anggap berbeda sehingga dihitung terpisah dalam dictionary. Hal yang sama juga berlaku untuk huruf kapital, seperti “who” dan “Who” yang di anggap berbeda. Untuk mengatasi hal ini, dapat digunakan method seperti `lower()`, `punctuation`, dan `translate()`, di mana `translate()` merupakan method yang paling halus untuk membersihkan teks.

```
line.translate(str.maketrans(fromstr, tostr, deletestr))
```

Fungsi `split()` memisahkan teks menjadi kata Berdasarkan spasi, sehingga kata seperti “soft!” dan “soft” dianggap berbeda, begitu juga “who” dan “Who”. Untuk menyamakan bentuk kata, dapat digunakan `lower()` dan `translate()` untuk membersihkan tanda baca.

```
>>> import string
>>> string.punctuation
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

Penggunaan program seperti dibawah ini :

```
1  import string
2
3  fname = input("Enter the file name: ")
4  try:
5      fhand = open(fname)
6  except:
7      print("File cannot be opened:", fname)
8      exit()
9
10 counts = dict()
11 for line in fhand:
12     line = line.rstrip()
13     line = line.translate(line.maketrans('', '', string.punctuation))
14     line = line.lower()
15     words = line.split()
16     for word in words:
17         if word not in counts:
18             counts[word] = 1
19         else:
20             counts[word] += 1
21
22 print(counts)
```

Outputnya :

```

Enter the file name: roméo-full.txt
('roméo': 40, 'and': 42, 'juliet': 32, 'act': 1, '2': 3, 'scene': 2, 'li': 1, 'capulets': 1, 'orchard': 2, 'enter': 1, 'he': 5, 'jests': 1, 'at': 9, 'scars': 1, 'that': 30, 'never': 2, 'felt': 1, 'a': 24, 'ward': 1, 'appears': 1, 'above': 6, 'window': 2, 'but': 18, 'soft': 1, 'what': 11, 'light': 5, 'through': 3, 'yonder': 2, 'breast': 1, 'it': 22, 'is': 21, 'the': 34, 'east': 1, 'sun': 2, 'arise': 1, 'fair': 4, 'kill': 2, 'envious': 2, 'moon': 4, 'she': 5, 'already': 1, 'sick': 2, 'pale': 1, 'with': 8, 'grief': 2, 'thou': 32, 'her': 14, 'maid': 2, 'art': 7, 'far': 2, 'more': 9, 'than': 6, 'he': 9, 'he': 14, 'not': 18, 'since': 1, 'vestal': 1, 'livery': 1, 'green': 1, 'none': 1, 'fools': 1, 'do': 2, 'woon': 1, 'cast': 1, 'off': 1, 'my': 29, 'lady': 2, 'n': 11, 'love': 24, 'know': 1, 'were': 9, 'speaks': 3, 'yet': 9, 'says': 1, 'nothing': 1, 'of': 20, 'eye': 2, 'discourses': 1, 'i': 61, 'will': 8, 'answer': 1, 'am': 7, 'too': 8, 'bold': 1, 'tis': 4, 'to': 34, 'we': 18, 'two': 1, 'fairest': 1, 'stars': 2, 'in': 11, 'all': 6, 'heaven': 3, 'having': 1, 'some': 3, 'business': 1, 'entreat': 1, 'eyes': 6, 'baldie': 1, 'their': 6, 'spheres': 1, 'till': 4, 'they': 5, 'return': 1, 'if': 12, 'there': 3, 'head': 2, 'brightness': 1, 'cheek': 4, 'would': 16, 'shame': 1, 'those': 2, 'as': 12, 'daylight': 1, 'doth': 2, 'lamp': 1, 'airy': 2, 'region': 1, 'stream': 1, 'wo': 18, 'height': 1, 'birds': 1, 'sing': 1, 'think': 2, 'night': 16, 'see': 2, 'how': 5, 'looms': 1, 'upon': 5, 'hand': 4, 'y low': 1, 'ought': 1, 'touch': 1, 'ay': 2, 'spak': 4, 'again': 6, 'angel': 1, 'for': 12, 'glorious': 3, 'this': 9, 'being': 2, 'oor': 1, 'winged': 1, 'messenger': 1, 'unto': 1, 'white-turned': 1, 'wandering': 1, 'mortals': 1, 'fall': 1, 'back': 4, 'gaze': 1, 'on': 4, 'him': 2, 'when': 2, 'bestrides': 1, 'lazing': 1, 'clouds': 1, 'sails': 1, 'bosom': 1, 'air': 1, 'wherefore': 2, 'dany': 2, 'thy': 28, 'father': 1, 'refuse': 1, 'name': 11, 'or': 4, 'will': 6, 'sworn': 1, 'ill': 8, 'no': 6, 'in rger': 1, 'capulet': 1, 'aside': 1, 'shall': 6, 'hear': 2, 'enemy': 2, 'thysell': 1, 'though': 1, 'montague': 5, 'whats': 2, 'nee': 5, 'foot': 2, 'arm': 1, 'face': 2, 'any': 4, 'other': 4, 'part': 2, 'belonging': 1, 'man': 2, 'which': 2, 'we': 2, 'call': 3, 'rose': 1, 'by': 14, 'small': 1, 'sweet': 8, 'callid': 1, 'retain': 1, 'dear': 7, 'perfection': 1, 'owes': 1, 'without': 1, 'title': 1, 'doff': 1, 'thee': 24, 'take': 3, 'myself': 2, 'word': 4, 'now': 1, 'baptized': 1, 'henceforth': 1, 'thus': 1, 'besceend': 1, 'stumblest': 1, 'counsel': 2, 'know': 1, 'tell': 1, 'saint': 2, 'hateful': 1, 'because': 1, 'an': 2, 'had': 1, 'written': 1, 'tear': 2, 'ears': 2, 'have': 13, 'drunk': 1, 'hundred': 1, 'words': 2, 'tongues': 2, 'utterance': 1, 'sound': 2, 'neither': 1, 'either': 1, 'dislike': 1, 'canest': 1, 'hither': 1, 'walls': 2, 'are': 3, 'high': 1, 'hard': 1, 'climb': 1, 'place': 2, 'death': 2, 'considering': 1, 'kissed': 2, 'find': 2, 'here': 4, 'loves': 1, 'wings': 1, 'did': 1, 'oerpec h': 1, 'these': 2, 'story': 1, 'limits': 1, 'cannot': 1, 'hold': 1, 'out': 2, 'can': 2, 'dawn': 1, 'attaght': 1, 'therefore': 1, 'let': 1, 'madder': 1, 'alack': 1, 'lies': 2, 'peril': 1, 'thine': 2, 'twenty': 2, 'swords': 1, 'look': 1, 'proof': 1, 'against': 1, 'omity': 1, 'world': 1, 'saw': 1, 'nights': 1, 'cloak': 1, 'hide': 1, 'from': 4, 'sight': 1, 'thee': 1, 'life': 1, 'better': 1, 'ended': 1, 'hate': 1, 'prorogued': 1, 'wanting': 1, 'whose': 1, 'direction': 1, 'foundst': 1, 'first': 1, 'p rompt': 1, 'inquire': 1, 'loot': 2, 'pilot': 1, 'wert': 1, 'vast': 1, 'shore': 1, 'wash': 1, 'farthest': 1, 'sea': 2, 'adventure': 1, 'such': 1, 'merchandise': 1, 'kn owl': 1, 'mask': 1, 'else': 1, 'maiden': 1, 'blush': 1, 'beapaint': 1, 'hast': 1, 'heard': 1, 'tonight': 1, 'fain': 1, 'dwell': 2, 'form': 1, 'spoke': 1, 'farewell': 1, 'complaint': 1, 'dost': 2, 'say': 5, 'nearest': 1, 'west': 2, 'prove': 4, 'false': 1, 'lovers': 2, 'perjaries': 1, 'then': 2, 'jow': 1, 'laugh': 1, 'gentle': 1, 'pronounce': 1, 'faithfully': 1, 'thinkest': 1, 'quickly': 1, 'won': 1, 'from': 1, 'perverse': 1, 'nay': 1, 'woo': 1, 'truth': 1, 'ford': 1, 'havior': 1, 'trust': 1, 'gentleman': 1, 'true': 1, 'cunning': 1, 'strange': 2, 'should': 2, 'been': 1, 'must': 1, 'confess': 1, 'overheardst': 1, 'ere': 2, 'was': 1, 'sare': 1, 'passion': 1, 'pardon': 1, 'lquote': 1, 'yielding': 1, 'dark': 1, 'hath': 1, 'discovered': 1, 'blessed': 1, 'saw': 6, 'tips': 1, 'silver': 1, 'fruittree': 1, 'tops': 1, 'inconstant': 1, 'monthly': 1, 'changes': 1, 'circled': 1, 'orb': 1, 'lest': 1, 'likewise': 1, 'variable': 1, 'gracious': 1, 'self': 1, 'god': 1, 'idolatri': 1, 'believe': 1, 'be acts': 1, 'well': 2, 'although': 1, 'joy': 2, 'contract': 1, 'rash': 1, 'unadvised': 1, 'sudden': 1, 'like': 1, 'lightning': 1, 'cease': 2, 'one': 2, 'lightens': 1, 'g ood': 9, 'bud': 1, 'summers': 1, 'ripening': 1, 'breath': 1, 'way': 2, 'beauteous': 1, 'flower': 1, 'nest': 1, 'meet': 1, 'repose': 1, 'rest': 2, 'come': 5, 'heart': 1, 'within': 5, 'breast': 2, 'leave': 2, 'unsatisfied': 1, 'satisfaction': 1, 'canst': 1, 'exchange': 1, 'faithful': 1, 'vow': 1, 'mine': 1, 'gave': 1, 'before': 1, 'd d': 1, 'request': 1, 'give': 1, 'wouldst': 1, 'withdraw': 1, 'purpose': 2, 'frank': 1, 'wish': 1, 'thing': 1, 'bounty': 1, 'boundless': 1, 'deep': 1, 'both': 1, 'inf into': 1, 'course': 4, 'calls': 2, 'noise': 1, 'adieu': 1, 'anon': 1, 'stay': 2, 'little': 2, 'exit': 4, 'aford': 1, 'dream': 1, 'flattering': 1, 'substantial': 1, 'reenter': 2, 'three': 1, 'indeed': 1, 'best': 1, 'honourable': 1, 'marriage': 1, 'send': 3, 'tomorrow': 3, 'procure': 1, 'where': 2, 'time': 1, 'perform': 1, 'rite': 1, 'fortunes': 1, 'lay': 1, 'follow': 1, 'lord': 1, 'throughout': 1, 'madam': 2, 'moribut': 1, 'meanst': 1, 'beseech': 1, 'sift': 1, 'thrive': 1, 'soul': 2, 'thous and': 2, 'times': 2, 'worse': 1, 'want': 1, 'goes': 1, 'toward': 2, 'schoolboys': 1, 'books': 1, 'school': 1, 'heavy': 1, 'looks': 1, 'retiring': 1, 'hist': 2, 'falcone rs': 1, 'voice': 1, 'lure': 1, 'fasselgentle': 1, 'bondage': 1, 'hoarse': 2, 'aloud': 1, 'cave': 1, 'echo': 1, 'make': 1, 'tongue': 1, 'repetition': 1, 'romeos': 1, 's ilversweet': 1, 'softest': 1, 'music': 1, 'attending': 1, 'oclock': 1, 'hour': 1, 'nine': 1, 'fall': 1, 'years': 1, 'forgot': 1, 'why': 1, 'stand': 2, 'remember': 1, 'forget': 2, 'still': 1, 'remembering': 1, 'company': 1, 'forgetting': 1, 'home': 1, 'almost': 1, 'morning': 1, 'gone': 1, 'further': 1, 'wantons': 1, 'bird': 2, 'lets': 1, 'hap': 1, 'poor': 1, 'prisoner': 1, 'his': 1, 'twisted': 1, 'gyves': 1, 'silk': 1, 'thread': 1, 'plucks': 1, 'lovingjealous': 1, 'liberty': 1, 'much': 1, 'cherish ing': 1, 'parting': 1, 'sorrow': 1, 'morrow': 1, 'sleep': 2, 'peace': 2, 'hence': 1, 'ghostly': 1, 'fathers': 1, 'cell': 1, 'help': 1, 'craw': 1, 'hap': 1)
PS D:\SEMESTER 2\PRAKTIKUM ALPRO\WAK ALPRO WEEK 11>

```

Link github : <https://github.com/maryonubatonis1-web/71251213-PrakALPRO-WEEK11.git>

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

```
data = {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}

print("key value item")
for key, value in data.items():
    print(key, value, key)
```

```
key value item
1 10 1
2 20 2
3 30 3
4 40 4
5 50 5
6 60 6
```

Penjelasan :

Program ini menampilkan isi dictionary dalam bentuk sederhana seperti table. Dengan `data.items()`, setiap pasangan key dan value diambil lalu ditampilkan menggunakan `for`. Outputnya berupa tiga kolom yaitu key, value, dan key lagi (diulang). Judul “key value item” hanya sebagai penanda tampilan.

SOAL 2

```
lista = ['red', 'green', 'blue']
listb = ['#FF0000', '#008000', '#0000FF']

result = dict(zip(lista, listb))

print(result)

{'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
```

Penjelasan :

Kode ini menggabungkan dua list menjadi dictionary dengan `zip()`. Lista menjadi key dan listb menjadi value, lalu `dict()` menyusunnya menjadi pasangan data.

SOAL 3

```
1  fname = input("Masukkan nama file : ")
2
3  try:
4      fhand = open(fname)
5  except:
6      print("File tidak bisa dibuka:", fname)
7      exit()
8
9  counts = {}
10
11 for line in fhand:
12     if line.startswith("From "):
13         words = line.split()
14         email = words[1]
15         counts[email] = counts.get(email, 0) + 1
16
17 print(counts)
```

```
Masukkan nama file : mbox-short.txt
{'stephen.marquard@uct.ac.za': 2, 'louis@media.berkeley.edu': 3, 'zqian@umich.edu': 4, 'rjlowe@iupui.edu': 2, 'cwen@iupui.edu': 5, 'gsilver@umich.edu': 3, 'wagnermr@iupui.edu': 1, 'antranig@caret.cam.ac.uk': 1, 'gopal.ramasammycook@gmail.com': 1, 'david.horwitz@uct.ac.za': 4, 'ray@media.berkeley.edu': 1}
```

Penjelasan :

Program ini membuka file yang dimasukan pengguna. Jika gagal, program berhenti. Jika berhasil, program mencari baris yang diawali "From", mengambil emailnya, lalu menghitung berapa kali email muncul menggunakan dictionary, kemudian menampilkan hasilnya.

SOAL 4

```
1  fname = input("Masukkan nama file : ")
2
3  try:
4      fhand = open(fname)
5  except:
6      print("File tidak bisa dibuka:", fname)
7      exit()
8
9  counts = {}
10
11 for line in fhand:
12     if line.startswith("From "):
13         words = line.split()
14         email = words[1]
15         domain = email.split("@")[1]
16         counts[domain] = counts.get(domain, 0) + 1
17
18 print(counts)
```

```
Masukkan nama file : mbox-short.txt
{'uct.ac.za': 6, 'media.berkeley.edu': 4, 'umich.edu': 7, 'iupui.edu': 8, 'caret.cam.ac.uk': 1, 'gmail.com': 1}
```

Penjelasan :

Program ini meminta nama file lalu membukanya. Jika berhasil, program membaca baris yang diawali "From", mengambil email, lalu mengekstrak domainnya. Setiap domain dihitung kemunculannya menggunakan dictionary, kemudian hasil akhirnya ditampilkan